Lista de Exercícios — Monte Carlo e Ajuste de Curvas com Minuit

F056 — Métodos Computacionais em Física de Altas Energias

Instruções gerais. Os exercícios devem ser resolvidos em Python. Cada problema deve ser acompanhado do código-fonte, das figuras geradas e de uma breve discussão dos resultados. A entrega é feita pelo seu repositório no GitHub, conforme descrito na Seção 1.

Parte I — Método de Monte Carlo

Problema 1 — Método de aceitação-rejeição

Transforme um gerador de números aleatórios uniformes já disponível em um gerador de números aleatórios distribuídos segundo uma PDF arbitrária. O usuário deve poder definir a função PDF(x) segundo a qual deseja gerar os números.

O método, chamado de *aceitação-rejeição*, funciona assim: gere um par de números aleatórios (x, y), com x uniforme em $[x_{\min}, x_{\max}]$ e y uniforme em $[0, f_{\max}]$, onde $f_{\max} \geq \max_x \mathrm{PDF}(x)$. Se $\mathrm{PDF}(x) > y$, o valor x é mantido como número aleatório gerado; caso contrário, o par é descartado e um novo par é sorteado.

Implemente o método como uma função reutilizável, seguindo o esqueleto abaixo. A função deve devolver, além das amostras aceitas, o número total de pares sorteados.

Observações.

- Use apenas números uniformes como *entrada*. Não use `numpy.random.normal` (ou equivalentes) para gerar a amostra.
- A função deve funcionar para qualquer PDF fornecida pelo usuário, e não apenas para a gaussiana do Problema 2. Teste-a também com pelo menos mais uma PDF à sua escolha (por exemplo, uma exponencial truncada ou $\mathrm{PDF}(x) \propto \sin^2 x$ em $[0, \pi]$).

Problema 2 — Teste do gerador com a distribuição normal

Teste o gerador do Problema 1 para gerar números aleatórios segundo a distribuição normal Gauss($\mu = 0, \sigma = 1$),

$$\mathrm{PDF}(x) = \frac{1}{\sqrt{2\pi}}\, e^{-x^2/2}.$$

(a) Quantos números aleatórios (pares (x, y)) você precisou gerar para obter os 10^4 números distribuídos segundo a PDF dada? Calcule a eficiência $\varepsilon = N_{\text{aceitos}}/N_{\text{sorteios}}$ e compare com o valor esperado

$$\varepsilon_{\text{esperada}} = \frac{\int_{x_{\min}}^{x_{\max}} \mathrm{PDF}(x)\, dx}{(x_{\max} - x_{\min})\, f_{\max}}.$$

(b) Repita para os intervalos $[-3, 3]$, $[-5, 5]$ e $[-10, 10]$. Como a eficiência varia? Repita o sorteio de 10^4 eventos 100 vezes para um dos intervalos e mostre a distribuição de N_{sorteios}.

```
def pdf_1(x):
     return Gaussian(x, 0.0, 1.0)

 def random_pdf(pdf, N, x_min, x_max):
```

```
    x_rand = []
    n_tries = 0
    while (len(x_rand) < N):
        n_tries += 2
        (x,y)= (random.uniform(x_min,x_max), random.uniform(0.0,1.0))
        if(pdf(x) > y):
            x_rand.append(x)
    print("I had to generate " + str(n_tries) + " random numbers to draw " +
        str(N) + " random numbers according to a given PDF.")
    return x_rand

N = 10000
x_rand = random_pdf(pdf_1, N, -5.0, 5.0)
x = np.linspace(-5,5,1000)

plt.title("Acceptance-rejection method")
plt.hist(x_rand, bins = 20, label="MC toys")
plt.plot(x, (N/2.5)*pdf_1(x), label="PDF")
plt.legend(loc="upper left")
plt.savefig('E1_Problem_34.pdf')
plt.clf()
    ...
```

Listing 1: Esqueleto sugerido para o Problema 1 e 2.

Parte II — Ajuste de curvas com Minuit

Contexto

No material de aula está disponível o programa `ajuste_higgs_v2.py`, que ajusta o espectro de massa de uma amostra simulada com o Pythia8 (arquivo `pythia_higgs.root`, árvore `events`, ramo `mH`) no intervalo $m \in [50, 400]$ GeV. O modelo é a soma de três componentes, cada uma normalizada no intervalo de análise:

- um sinal gaussiano, $f_s(m; \mu, \sigma)$;
- um *background* do tipo Breit-Wigner com pico fixo em 91 GeV, $f_{BW}(m; \Gamma)$;
- um *background* exponencial decrescente, $f_{\exp}(m; \tau) \propto e^{-(m-m_{\min})/\tau}$.

Os rendimentos de cada componente, s, b_{BW} e $b_{\exp}$, também são parâmetros livres, de modo que o modelo (em eventos por unidade de massa) é

$$\nu(m) = s\, f_s(m; \mu, \sigma) + b_{BW}\, f_{BW}(m; \Gamma) + b_{\exp}\, f_{\exp}(m; \tau).$$

Dois ajustes são feitos com o Minuit (via `iminuit`):

(i) Máxima verossimilhança estendida (dados não binados), minimizando

$$-\ln L = (s + b_{BW} + b_{\exp}) - \sum_{j=1}^{N} \ln \nu(m_j),$$

(ii) Qui-quadrado (dados binados em N_{bins} intervalos de largura Δm), minimizando

$$\chi^2 = \sum_{i=1}^{N_{\text{bins}}} \frac{(n_i - \nu(m_i)\,\Delta m)^2}{\sigma_i^2}, \qquad \sigma_i^2 = \max(n_i, 1),$$

onde n_i é a contagem e m_i o centro do i-ésimo bin.

Os trechos abaixo mostram as partes principais do código. Leia o programa completo antes de começar.

```
ARQUIVO_ROOT = "pythia_higgs.root"
NOME_ARVORE = "events"
NOME_RAMO = "mH"

MMIN = 50.0
MMAX = 400.0

NBINS = 1000

# Pico fixo do background Breit-Wigner em GeV
BW_MU = 91.0
```

Listing 2: Configurações do programa (`ajuste_higgs_v2.py`).

```
def nll(mu, sigma, bw_gamma, tau, s, b_bw, b_exp):

    sinal = fs(dados, mu, sigma)
    background_bw = fbw(dados, bw_gamma)
    background_exp = fexp(dados, tau)

    densidade = (
        s * sinal
        + b_bw * background_bw
        + b_exp * background_exp
    )

    densidade = np.maximum(densidade, 1.e-300)

    rendimento_total = s + b_bw + b_exp

    return rendimento_total - np.sum(np.log(densidade))
```

Listing 3: Função de custo da máxima verossimilhança estendida.

```
previsao = (
    s * sinal + b_bw * background_bw + b_exp * background_exp
) * largura_bin

variancia = np.maximum(contagens, 1.0)

residuos = contagens - previsao

return np.sum(residuos**2 / variancia)

# ...

fit = Minuit(chi2_fit, mu=92.0, sigma=4.0, bw_gamma=5.0, tau=50.0,
             s=0.27 * n_total, b_bw=0.30 * n_total, b_exp=0.43 * n_total)

fit.limits["sigma"] = (1.e-4, 50.0)
# ... (demais limites)

# Para chi-quadrado: Delta chi2 = 1 corresponde a 1 sigma.
fit.errordef = Minuit.LEAST_SQUARES

fit.migrad()
fit.hesse()
```

Listing 4: Função de custo do qui-quadrado e configuração do Minuit.

Problema 3 — Melhorando o ajuste

Modifique o programa `ajuste_higgs_v2.py` do diretório `Exemplos` para obter um ajuste melhor ou mais informativo. Salve a sua versão em um novo arquivo (`ajuste_higgs_melhorado.py`), sem sobrescrever o original. Você deve inspecionar o código criticamente. Em particular, o programa original apresenta os seguintes pontos de atenção, que devem ser corrigidos ou discutidos na sua versão:

(1) **Variância do qui-quadrado:** com `NBINS = 1000` em 350 GeV, os bins têm largura de 0.35 GeV e poucas contagens. O uso de $\sigma_i^2 = \max(n_i, 1)$ nessas condições pode enviesar o ajuste (o peso de cada bin depende da própria flutuação estatística). Discuta e proponha uma alternativa (ver itens (b) e (c) abaixo).

(2) **Valores iniciais diferentes nos dois ajustes:** o valor inicial de μ é 125 GeV no ajuste de máxima verossimilhança e 92 GeV no de qui-quadrado. Como o valor 92 GeV está sobre o pico da Breit-Wigner (91 GeV), o Minuit pode convergir para um mínimo diferente. Use valores iniciais consistentes e verifique se o resultado depende deles.

(3) **Falta de qualidade do ajuste:** o programa não imprime χ^2/ndof nem p-valor (ver item obrigatório (a)), e só emite um aviso quando `fit.valid` é falso, sem interromper a análise.

(4) Estude a dependência dos resultados com o número de bins: repita o ajuste de qui-quadrado para pelo menos cinco valores de `NBINS` entre 50 e 1000 e apresente uma tabela (e um gráfico) com μ, σ e s, e suas incertezas, em função de `NBINS`, comparando com o resultado da máxima verossimilhança (que não depende de `NBINS`). Discuta o que você observa.

(5) Tornar a posição do pico da Breit-Wigner (`BW_MU`) um parâmetro livre e comparar com o valor fixo.

(6) Propor e testar outra forma para o sinal (por exemplo, uma *Crystal Ball* ou uma soma de duas gaussianas) ou para o *background*.

1 Entrega

A entrega deve ser feita na área do GitHub que vocês já possuem, em um diretório dedicado a esta lista, com a seguinte organização mínima:

```
lista_montecarlo_ajuste/
|-- README.md
|-- problema1_2_aceita_rejeita.py
|-- ajuste_higgs_melhorado.py    # sua versão do Problema 4
`-- figuras/
    |-- gauss_aceita_rejeita.png
    |-- ajuste_original.png
    `-- ajuste_melhorado.png
```

Listing 5: Estrutura sugerida do diretório de entrega.

O arquivo `README.md` deve conter:

- instruções para executar cada programa (e as dependências necessárias: `numpy`, `scipy`, `matplotlib`, `iminuit` e PyROOT);
- as respostas às perguntas de cada problema, incluindo os números pedidos (por exemplo, o número de sorteios do Problema 2);
- as figuras geradas, incluídas com a sintaxe do Markdown;

Não inclua o arquivo `pythia_higgs.root` no repositório.